

Messages > Model capabilities

Embeddings

Text embeddings are numerical representations of text that enable measuring semantic similarity. This guide introduces embeddings, their applications, and how to use embedding models for tasks like search, recommendations, and anomaly detection.

 Copy page 

Before implementing embeddings

When selecting an embeddings provider, there are several factors you can consider depending on your needs and preferences:

- **Dataset size & domain specificity:** size of the model training dataset and its relevance to the domain you want to embed. Larger or more domain-specific data generally produces better in-domain embeddings
- **Inference performance:** embedding lookup speed and end-to-end latency. This is a particularly important consideration for large scale production deployments
- **Customization:** options for continued training on private data, or specialization of models for very specific domains. This can improve performance on unique vocabularies

How to get embeddings with Anthropic

Anthropic does not offer its own embedding model. One embeddings provider that has a wide variety of options and capabilities encompassing all of the preceding considerations is Voyage AI.

Voyage AI makes state-of-the-art embedding models and offers customized models for specific industry domains such as finance and healthcare, or bespoke fine-tuned models for individual customers.

The rest of this guide is for Voyage AI, but you should assess a variety of embeddings vendors to find the best fit for your specific use case.

Available models

Voyage recommends using the following text embedding models:

Voyage 4 (latest generation)

voyage-4-large	32,000	1024 (default), 256, 512, 2048	The best general-purpose and multilingual retrieval quality. See the Voyage 4 blog post for details.
voyage-4	32,000	1024 (default), 256, 512, 2048	Optimized for general-purpose and multilingual retrieval quality. Balances quality and efficiency. See the Voyage 4 blog post for details.
voyage-4-lite	32,000	1024 (default), 256, 512, 2048	Optimized for latency and cost. See the Voyage 4 blog post for details.
voyage-4-nano	32,000	1024 (default), 256, 512, 2048	Open-weight model (Apache 2.0 license) available on Hugging Face. See the Voyage 4 blog post for details.

Previous generation

Model	Context length	Embedding dimension	Description
voyage-3-large	32,000	1024 (default), 256, 512, 2048	The best general-purpose and multilingual retrieval quality. See the voyage-3-large blog post for details.
voyage-3.5	32,000	1024 (default), 256, 512, 2048	Optimized for general-purpose and multilingual retrieval quality. See the voyage-3.5 blog post for details.
voyage-3.5-lite	32,000	1024 (default), 256, 512, 2048	Optimized for latency and cost. See the voyage-3.5 blog post for details.
voyage-code-3	32,000	1024 (default), 256, 512, 2048	Optimized for code retrieval. See the voyage-code-3 blog post for details.

voyage-finance-2	32,000	1024	Optimized for finance retrieval and RAG. See the voyage-finance-2 blog post for details.
voyage-law-2	16,000	1024	Optimized for legal and long-context retrieval and RAG. Also improved performance across all domains. See the voyage-law-2 blog post for details.

Additionally, Voyage recommends the following multimodal embedding models:

Model	Context length	Embedding dimension	Description
voyage-multimodal-3.5	32,000	1024 (default), 256, 512, 2048	Rich multimodal embedding model that can vectorize interleaved text, images, and videos. Includes video support as the first production-grade video embedding model. See the voyage-multimodal-3.5 blog post for details.
voyage-multimodal-3	32,000	1024	Rich multimodal embedding model that can vectorize interleaved text and content-rich images, such as screenshots of PDFs, slides, tables, figures, and more. See the voyage-multimodal-3 blog post for details.

The following contextualized chunk embedding models produce chunk-level vectors that capture full document context without manual metadata augmentation. Call these models with `contextualized_embed()` instead of `embed()` :

Model	Context length	Embedding dimension	Description
voyage-context-4	120,000	1024 (default), 256, 512, 2048	Contextualized chunk embeddings optimized for general-purpose and multilingual retrieval quality. See the voyage-context-4 blog post for details.
voyage-context-3	120,000	1024 (default), 256, 512, 2048	Contextualized chunk embeddings optimized for general-purpose and multilingual retrieval quality. See the voyage-context-3 blog post for details.

Model	Context length	Description
rerank-2.5	32,000	Highest accuracy. Recommended for most applications. See the rerank-2.5 blog post for details.
rerank-2.5-lite	32,000	Optimized for latency and cost. See the rerank-2.5 blog post for details.

Need help deciding which text embedding model to use? Check out the [Voyage AI FAQ](#).

Getting started with Voyage AI

To access Voyage embeddings:

1. Sign up on Voyage AI's website.
2. Obtain an API key.
3. Set the API key as an environment variable for convenience:

```
export VOYAGE_API_KEY="<your secret key>"
```



You can obtain the embeddings by either using the official [voyageai Python package](#) or HTTP requests, as described in the following sections.

Voyage Python library

Install the `voyageai` package using the following command:

```
pip install -U voyageai
```



Then, you can create a client object and start using it to embed your texts:

```
vo = voyageai.Client()
# This will automatically use the environment variable VOYAGE_API_KEY.
# Alternatively, you can use vo = voyageai.Client(api_key="<your secret key>")

texts = ["Sample text 1", "Sample text 2"]

result = vo.embed(texts, model="voyage-4", input_type="document")
print(result.embeddings[0])
print(result.embeddings[1])
```

`result.embeddings` is a list of two embedding vectors, each containing 1024 floating-point numbers. After running the preceding code, the two embeddings are printed on the screen:

```
[-0.013131560757756233, 0.019828535616397858, ...] # embedding for "Sample text 1"
[-0.0069352793507277966, 0.020878976210951805, ...] # embedding for "Sample text 2"
```

When creating the embeddings, you can specify a few other arguments to the `embed()` function.

For more information on the Voyage Python package, see the [Voyage Python package documentation](#).

Voyage HTTP API

You can also get embeddings by requesting Voyage HTTP API. For example, you can send an HTTP request through the `curl` command in a terminal:

cURL

```
curl https://api.voyageai.com/v1/embeddings \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $VOYAGE_API_KEY" \
-d '{
  "input": ["Sample text 1", "Sample text 2"],
  "model": "voyage-4"
}'
```

The response you would get is a JSON object containing the embeddings and the token usage:

```
"data": [  
  {  
    "embedding": [-0.013131560757756233, 0.019828535616397858 /* ... */],  
    "index": 0  
  },  
  {  
    "embedding": [-0.0069352793507277966, 0.020878976210951805 /* ... */],  
    "index": 1  
  }  
],  
"model": "voyage-4",  
"usage": {  
  "total_tokens": 10  
}
```

For more information on the Voyage HTTP API, see the [Voyage HTTP API documentation](#).

AWS Marketplace

Voyage embeddings are available on [AWS Marketplace](#). Instructions for accessing Voyage on AWS are available in the [Voyage AWS Marketplace documentation](#).

Quickstart example

The following brief example shows how to use embeddings.

Suppose you have a small corpus of six documents to retrieve from

```
documents = [  
  "The Mediterranean diet emphasizes fish, olive oil, and vegetables,  
  believed to reduce chronic diseases.",  
  "Photosynthesis in plants converts light energy into glucose and produces  
  essential oxygen.",  
  "20th-century innovations, from radios to smartphones, centered on  
  electronic advancements.",  
  "Rivers provide water, irrigation, and habitat for aquatic species, vital  
  for ecosystems.",  
  "Apple's conference call to discuss fourth fiscal quarter results and  
  business updates is scheduled for Thursday, November 2, 2023 at 2:00 p.m. PT /  
  5:00 p.m. ET.",  
  "Shakespeare's works, like 'Hamlet' and 'A Midsummer Night's Dream,' endure  
  in literature.",  
]
```



```
import voyageai

vo = voyageai.Client()

# Embed the documents
doc_embs = vo.embed(documents, model="voyage-4",
input_type="document").embeddings
```



The embeddings allow you to do semantic search / retrieval in the vector space. Given an example query,

```
query = "When is Apple's conference call scheduled?"
```



Next, convert it into an embedding and conduct a nearest neighbor search to find the most relevant document based on the distance in the embedding space.

```
import numpy as np

# Embed the query
query_embd = vo.embed([query], model="voyage-4",
input_type="query").embeddings[0]

# Compute the similarity
# Voyage embeddings are normalized to length 1, therefore dot-product
# and cosine similarity are the same.
similarities = np.dot(doc_embs, query_embd)

retrieved_id = np.argmax(similarities)
print(documents[retrieved_id])
```



Note that `input_type="document"` and `input_type="query"` are used for embedding the document and query, respectively. More specification can be found in [Voyage Python library](#).

The output is the fifth document, which is indeed the most relevant to the query:

```
Apple's conference call to discuss fourth fiscal quarter results and business
updates is scheduled for Thursday, November 2, 2023 at 2:00 p.m. PT / 5:00 p.m.
ET.
```



If you are looking for a detailed set of recipes on how to do RAG with embeddings, including vector databases, check out the [RAG recipe](#).

- Why do Voyage embeddings have superior quality? >
- What embedding models are available and which should I use? >
- Which similarity function should I use? >
- What is the relationship between characters, words, and tokens? >
- When and how should I use the input_type parameter? >
- What quantization options are available? >
- How can I truncate Matryoshka embeddings? >

Pricing

Visit Voyage's [pricing page](#) for the most up to date pricing details.

Cette page vous a-t-elle été utile ?



K-12 teachers

Life sciences

Partners

Claude on AWS

Claude on Google Cloud

Help and security

Availability

Status

Support

Discord

Terms and policies

Privacy policy

Responsible disclosure policy

Terms of service: Commercial

Terms of service: Consumer

Usage policy

Powered by Claude

Service partners

Startups program

Company

Anthropic

Careers

Economic Futures

Research

News

Responsible Scaling Policy

Security and compliance

Transparency